

Practical No.:	03
Practical Title :	To understand Continuous Integration, install and configure Jenkins with Maven/Ant/Gradle to setup a build Job.
Aim:	To understand Continuous Integration by installing and configuring Jenkins with build tools like Maven/Ant/Gradle.
Objective:	To study the concepts of Continuous Integration (CI) in the software development lifecycle. To install Jenkins and configure it with Maven for automating build jobs.

Theory:**What is Continuous Integration (CI)?**

Continuous Integration is a DevOps software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run. The key goals of CI are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

What is Jenkins?

Jenkins is an open-source automation server that helps automate the parts of software development related to building, testing, and deploying, facilitating continuous integration and continuous delivery. It is a server-based system that runs in servlet containers such as Apache Tomcat.

What is Maven?

Maven is a powerful project management and comprehension tool used primarily for Java projects. It helps in managing project builds, reporting, and documentation from a central piece of information. Maven simplifies the build process by providing a uniform build system and managing dependencies automatically.

Benefits of Continuous Integration:

Early Bug Detection: By integrating frequently, teams can detect errors and conflicts much earlier in the development cycle.

- **Improved Collaboration:** CI encourages developers to share their code more often, leading to better team communication.
- **Faster Release Cycles:** Automated building and testing allow for quicker delivery of features to the end users.
- **Reduced Manual Effort:** Automation reduces the risk of human error during the build and deployment process

Procedure:**Steps for Installation and Configuration:****Step 1: Install and Configure Java Development Kit (JDK)**

Download the JDK (Version 17, 21, or 25) from the official Oracle or OpenJDK website.

<https://www.oracle.com/in/java/technologies/downloads/#jdk25-windows>

Run the installer and follow the setup wizard to complete the installation.

Set Environment Variables:

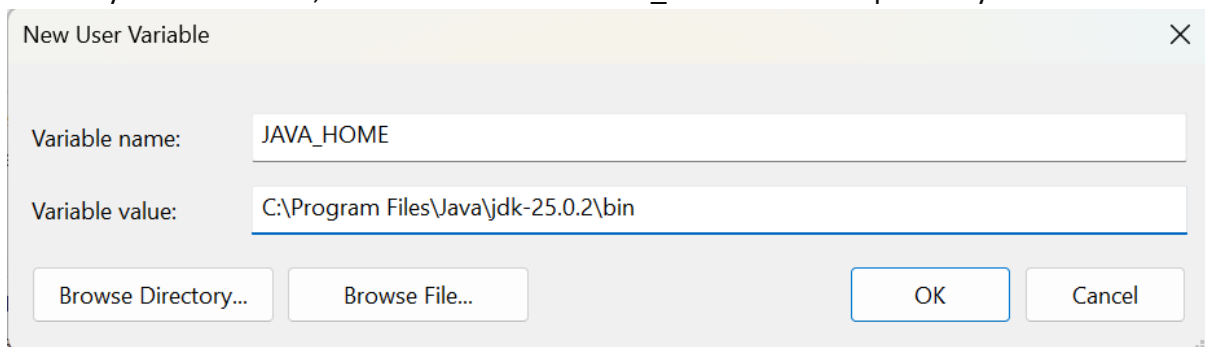
Open File Explorer and navigate to the folder where you installed Java (typically C:\Windows\Program Files\Java).

Open the JDK folder -> Open bin folder.

Click on the Address Bar at the top of the File Explorer window and copy the path.

Go to System Properties > Environment Variables.

Under System Variables, click 'New' and add JAVA_HOME with the path to your JDK folder.



Verify by typing `java -version` in the command prompt.

```
C:\Users>java --version
java 25.0.2 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)
```

Step 2: Install and Initialize Jenkins

Download the Jenkins Windows installer (.msi) or Generic Java package (.war).

<http://jenkins.io/download/>

Visit to windows to Install Jenkins

Jenkins  [Blog](#) [Success Stories](#) [Contributor Spotlight](#) [Documentation](#) [Plugins](#) [Community](#) [Subprojects](#) [Security](#) [About](#) [Support](#) [Download](#)  

4. You may also want to verify the package you downloaded. [Learn more about verifying Jenkins downloads.](#)

Download Jenkins 2.541.2 LTS for:

 Generic Java package (.war) SHA-256: dc9d532e54d4b7e7d78edcd3217876dd4811b49d1a3b66e599ad4a642d57193 	
 Docker	
 Kubernetes	
 Ubuntu/Debian	
 Red Hat Enterprise Linux and derivatives	
 Fedora	
 Windows	
 openSUSE	
 FreeBSD	Third party
 Gentoo	Third party
 macOS	Third party

Download Jenkins 2.552 for:

 Generic Java package (.war) SHA-256: 028dcd160d08a7acd605f4d1326429eba3313c33481fd07976eeeb162f0c52e 	
 Docker	
 Ubuntu/Debian	
 Red Hat Enterprise Linux and derivatives	
 Fedora	
 Windows	
 openSUSE	
 Arch Linux	Third party
 FreeBSD	Third party
 Gentoo	Third party
 macOS	Third party

Complete the installation process as shown below:

Jenkins 2.551 Setup 

Service Logon Credentials

Enter service credentials for the service.

Jenkins 2.551 installs and runs as an independent Windows service. To operate in this manner, you must supply the user account credentials for Jenkins 2.551 to run successfully.

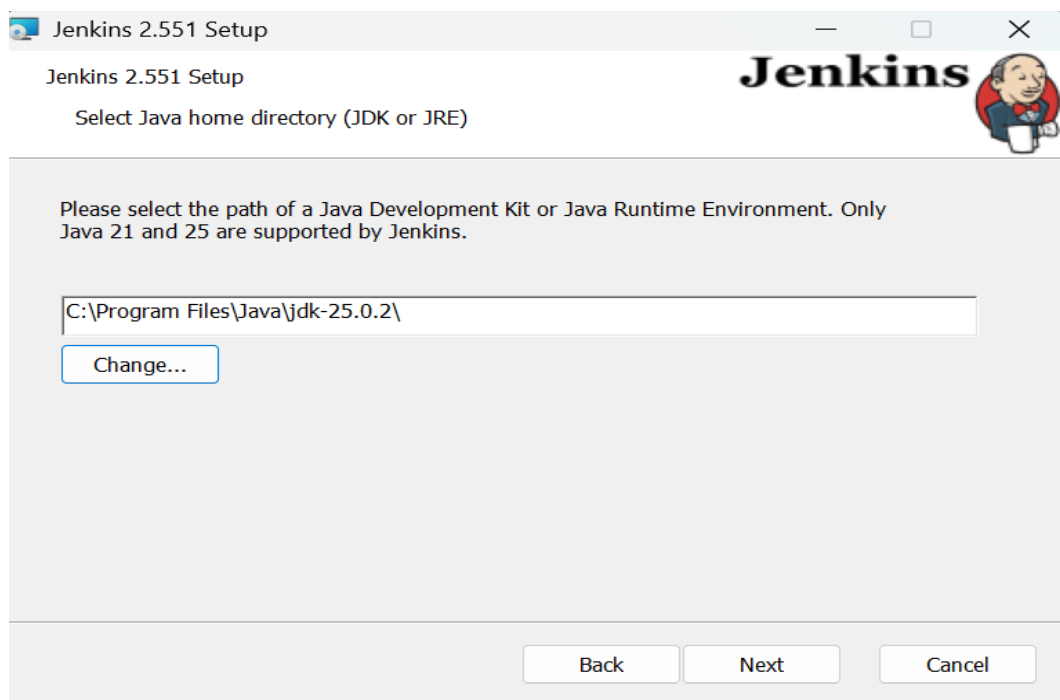
Logon Type:

☒ Run service as LocalSystem (not recommended)

☐ Run service as local or domain user:

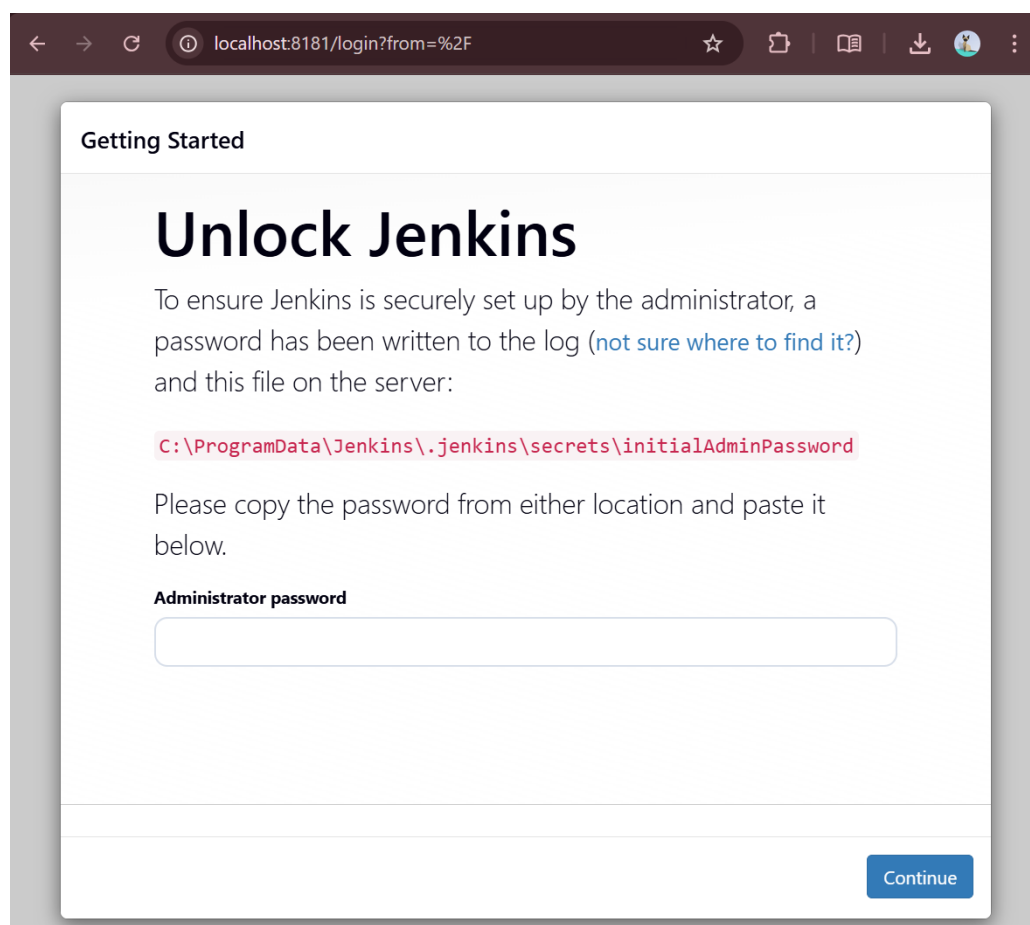
Account:

Password:



Next -> Next -> Install -> Finish.

Open your browser and go to <http://localhost:8181>.



Unlock Jenkins: Copy the file path displayed on the screen in file explorer and open the file in Notepad (Save this file for future use).

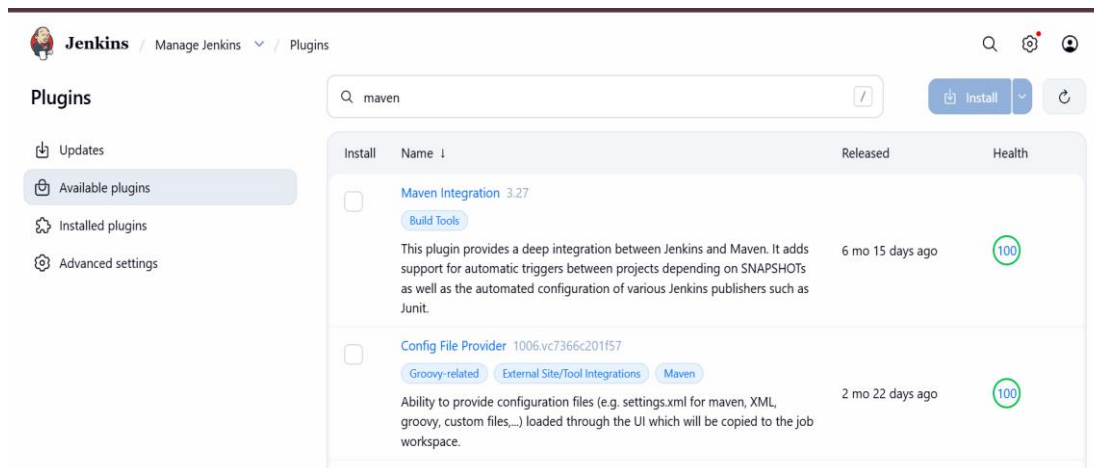
Select "Install suggested plugins" and wait for the process to complete.

Skip the Admin User form and continue as a admin.

Step 3: Configure Maven in Jenkins

From the Jenkins Dashboard, go to Manage Jenkins > Plugins.

Go to the Available Plugins tab, search for "Maven Integration", and click Install.



Step 4: Prepare GitHub Repository

Log in to your GitHub account and create a new repository named maven.

Inside the repository, create two files:

pom.xml

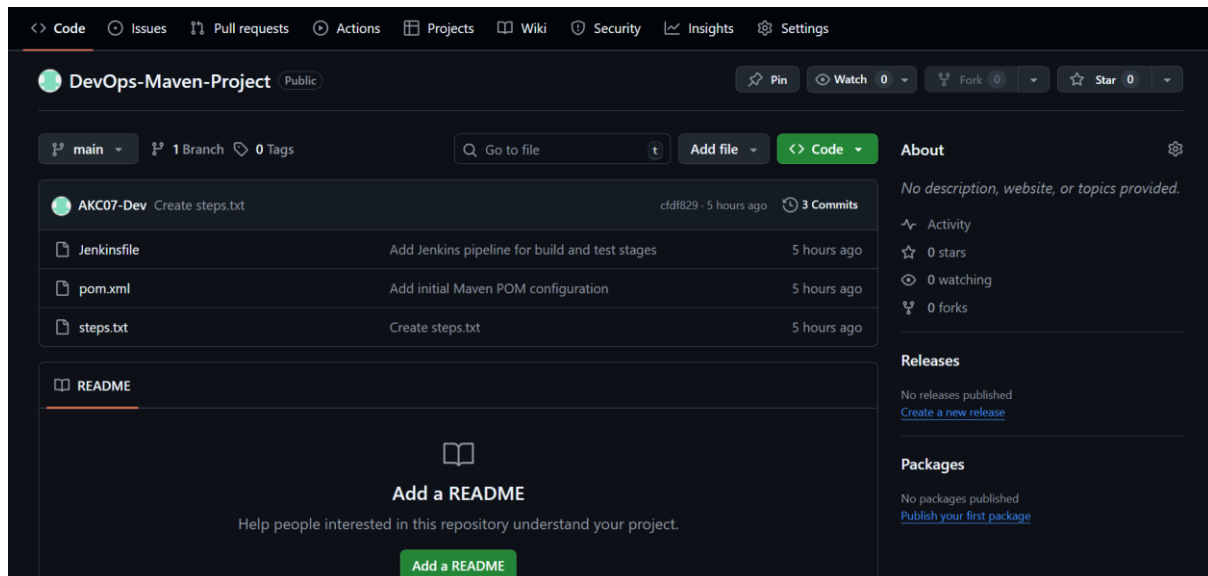
Jenkinsfile

Copy the code for these files from:

<https://github.com/AKC07-Dev/DevOps-Maven-Project/tree/main>

Commit and save these files in the repository.

Copy the Repository URL.



Step 5: Create and Run a Jenkins Build Job

On the Jenkins Dashboard, click on New Item.

Enter the item name (e.g., MavenProject), select Pipeline, and click OK.

New Item

Enter an item name

Maven

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

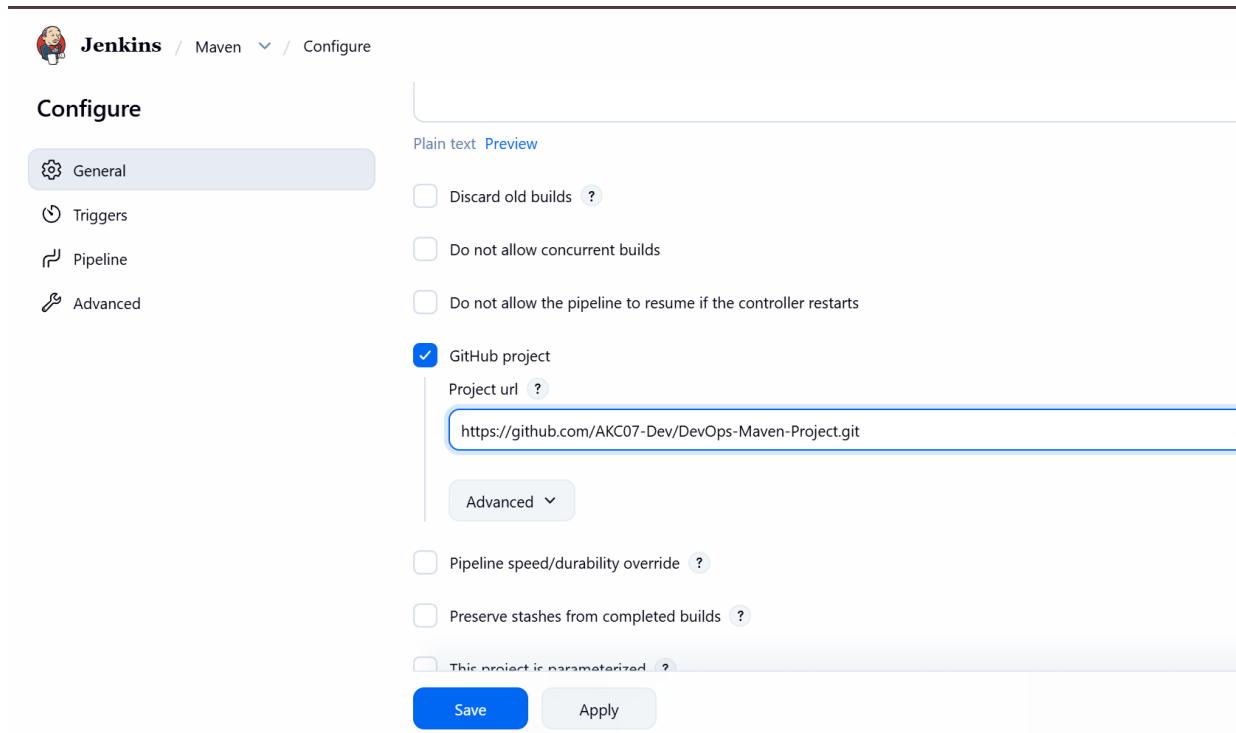
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

Select Git as the SCM and paste your GitHub Repository URL.



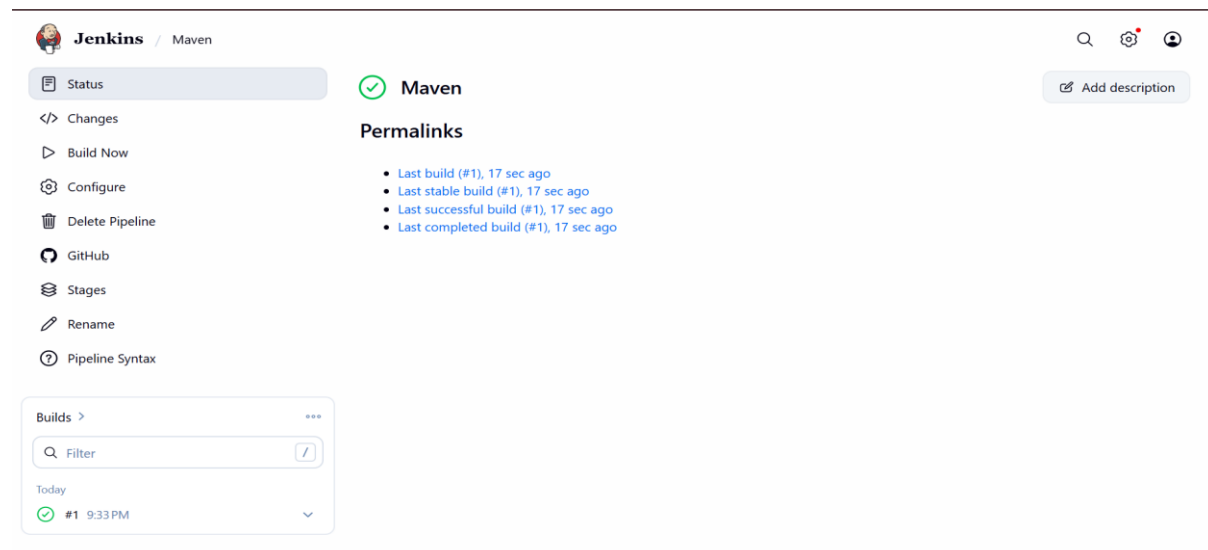
The image shows the Jenkins configuration page for a Maven project. The left sidebar contains the following menu items: Jenkins, Maven, and Configure. The main content area is titled 'Configure' and has a 'General' tab selected. The 'General' tab contains the following options:

- ☐ Discard old builds ?
- ☐ Do not allow concurrent builds
- ☐ Do not allow the pipeline to resume if the controller restarts
- ☒ GitHub project

The 'GitHub project' section is expanded, showing the 'Project url' field with the value `https://github.com/AKC07-Dev/DevOps-Maven-Project.git`. Below the 'Project url' field is an 'Advanced' dropdown menu. At the bottom of the configuration page are 'Save' and 'Apply' buttons.

Click Save.

Click Build Now from the left sidebar.



The image shows the Jenkins dashboard for the Maven project. The left sidebar contains the following menu items: Jenkins, Maven, Status, Changes, Build Now, Configure, Delete Pipeline, GitHub, Stages, Rename, and Pipeline Syntax. The main content area is titled 'Maven' and has a green checkmark icon. Below the title is a 'Permalinks' section with the following links:

- Last build (#1), 17 sec ago
- Last stable build (#1), 17 sec ago
- Last successful build (#1), 17 sec ago
- Last completed build (#1), 17 sec ago

At the bottom of the dashboard is a 'Builds' section with a filter input field and a list of builds. The first build is shown with a green checkmark icon and the text '#1 9:33 PM'.

Step 6: Verify the Build

Click on the build number in the Build History.

Select Console Output.

If the log ends with "FINISHED: SUCCESS", your Continuous Integration setup is working correctly.

 **Jenkins** / Maven ▾ / #1 ▾

 Status

 </> Changes

 Console Output

 Edit Build Information

 Delete build '#1'

 Timings

 Pipeline Overview

 Replay

Console

 Download

 Copy

 View as plain text

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion:
